Artificial Intelligence Technology and Application

Python
Lab Guide

Student Version

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD.

Contents

1 Data Type Experiment 1
1.1 Procedure 1
1.1.1 Hello world 1
1.1.2 Data Type: Number 1
1.1.3 Data Type: String 1
1.1.4 Data Type: List 2
1.1.5 Data Type: Tuple 3
1.1.6 Data Type: Dictionary 4
1.1.7 Data Type: Set 4
2 Control Loop Experiment 6
2.1 Procedure 6
2.1.1 Deep Copy and Shallow Copy 6
2.1.2 if Statement 6
2.1.3 Loop Statement 7
3 Function and Object-oriented Experiment 9
3.1 Procedure 9
3.1.1 Function Customization 9
3.1.2 Object Orientation 10
4 Standard Library Usage 12
4.1 Procedure 12
4.1.1 Standard Library Usage 12

1 Data Type Experiment

1.1 Procedure

1.1.1 Hello world

The first Python program generates "hello world".

```
# Generate "hello world".

# Generate "hello world". The output is the same when single and double quotation marks are carried in input.
```

1.1.2 Data Type: Number

You need to be familiar with the basic operations of numbers in Python. Note that Boolean operations in Python use the keyword and/or/not instead of the operator.

```
# The output is 1. By default, True indicates 1, and False indicates 0.

# If True is displayed, enter "or" or perform the OR operation.

# The output is 2, and // is the rounding operator.

# The output is 1, and % is the modulo operator.

# The output is 9, and ** indicates the power operation.

# The output is 6.6. By default, the sum of numbers of different precisions is the number of the highest precision type.
```

1.1.3 Data Type: String

Step 1 Basic operations on strings:

```
# Assign value "python" to variable S.

# len(obj): Return the object length.
# Output: 6

# The output is pyn. Elements are obtained by index.
```

```
# The output is python1 pythonpython, which indicates mergence and duplication.
```

Step 2 Immutability of strings:

```
# Assign value "python" to variable S.

# The program is abnormal.

# New string Zython is generated and assigned to S1.

# Output: S:python, S1:Zython
```

Step 3 Common operations on strings:

```
# Assign value "python" to variable S.

# str.split(str="", num=-1): The string is split by separator. If the num parameter has a value, divide the string into num+1 substrings. The value -1 indicates that all strings are split.

# The output is ['pyt','on']. The string is split by h.

# str.replace(old, new[, max]): A string generated after the old string is replaced with the new string is returned. If the third parameter max is specified, the number of times that the old string is replaced with the new string cannot exceed the value of max.
# In the string, py is replaced with PY.

# str.upper(): Return the value after lowercase letters are converted to uppercase letters.
# PYTHON

# str.lower(): Return the value after uppercase letters are converted to lowercase letters.
# The output is python because all uppercase letters are converted to lowercase letters.

# \n is a newline character.

# str.join(sequence): sequence indicates a sequence to be joined. In the output, the new string generated after the elements in the specified character join sequence is returned.
# The output is life is short. The join function is used for concatenating strings.

# Format the string.

# Output: hello world 12
```

1.1.4 Data Type: List

Common operations on lists:

```
animals = ['cat', 'dog', 'monkey']
# list.append(obj): Add a new object to the end of the list.
# Append an element.
```

```
# Output: ['cat', 'dog', 'monkey', 'fish']

# list.remove(obj): Remove the first match for a value in the list.
# Delete element fish.

# Output: ['cat', 'dog', 'monkey']

# list.insert(index, obj): Insert a specified object to a specified position in the list. The index indicates the position.
# Insert element fish at subscript 1.

# Output: ['cat', 'fish', 'dog', 'monkey']

# list.pop([index=-1]): Remove the element (the last element by default) corresponding to the subscript in the list. The index indicates the subscript.
# Delete the element whose subscript is 1.

# Output: ['cat', 'dog', 'monkey']

# Traverse and obtain the elements and indexes.
# enumerate(sequence): Return an index sequence consisting of a data object that can be traversed and list the data and subscripts. This function is usually used in the for loop.
# Index consisting of the element subscript and element


Output: (0, cat)
    (1, dog)
    (2, monkey)
# List derivation.
# Generate a list of elements that comply with rules in batches.

#['catcat ', 'dogdog ', 'monkeymonkey ']


# list.sort(cmp=None, key=None, reverse=False): The cmp parameter is an optional parameter. If this parameter is specified, the method uses this parameter is used for sorting. Key is an element used for comparison. reverse indicates the sorting rule, and False indicates the ascending order.
# Sort the list.

# Output: [12,32,45,55]

# list.reverse(): Element in the reverse list.
# Reverse the list.

# Output: [55,45,32,12]
```

1.1.5 Data Type: Tuple

Common operations on tuples:

```
# Create a tuple.

#Tuples are combined. The output is (1, 2, 3, 4, 5).
```

```
# A tuple with only one element, which is different from a number.

# Create a tuple.

# The program is abnormal, and the tuple is unchangeable.

# Elements that can be changed in a tuple are changeable.

# (12,45,32,55,[2,0,3])
```

1.1.6 Data Type: Dictionary

Common operations on dictionaries:

```
# Three value assignment operations on dictionaries.



# dict.copy(): Copy data.


# {'food':'Spam','quantity':4,'color':'pink'}

# {'food':'Spam','quantity':4,'color':'red'}

# Element access.
# Obtain the error information.

# Output: None

# Output: The key value does not exist.

# Output: dict_keys(['food', 'quantity', 'color'])

# Output: dict_values(['Spam', 4, 'red'])


# Output: dict_items([('food', 'Spam'), ('quantity', 4), ('color', 'red')])
# Clear all data in the dictionary.

# Output: {}

# Delete the dictionary.

# The program is abnormal, and a message is displayed, indicating that d is not defined.
```

1.1.7 Data Type: Set

Common operations on sets:

```
sample_set = {'Prince', 'Techs'}
# The output is False. in is used to check whether an element exists in the set.
```

```
# set.add(obj): Add an element to a set. If the element to be added already exists in the set, no operation is performed.
# Add element Data to the set.

# Output: {'Prince', 'Techs', 'Data'}

# Output: 3

# set.remove(obj): Remove a specified element from a set.
# Delete element Data.

# {'Prince', 'Techs'}

list2 = [1,3,1,5,3]
# The output is [1,3,5]. The uniqueness of the set elements is used to deduplicate the list.

# Invariable set.
```

2 Control Loop Experiment

2.1 Procedure

2.1.1 Deep Copy and Shallow Copy

The copy module in Python is used to implement deep copy.

```
import copy
# Create a dictionary.


# Shallow copy.


# Deep copy.


# Change the value of the nested list in the original data.
```

Output:

```
Dict1:{'name':'lee', 'age':89, 'num':[1,6,8]}
# The shallow copy data is modified.

# The deep copy data is not modified.
```

2.1.2 if Statement

You can use the if statement in Python to determine the level of a score input by a user.

```
# Determine the entered score.
# input(): Receive input data.

# The input function receives input, which is a string.


# Convert the score to a number.
# try:    except Exception: ... is a Python statement used to capture exceptions. If an error occurs in the statement in try, the statement in except is executed.
```

```
# Check whether the entered value is greater than the score of a level.

# Generate the level when conditions are met.
```

2.1.3 Loop Statement

Step 1 for loop

Use the for loop to generate a multiplication table.

```
# Define the outer loop.

# Define the inner loop.

# Format the output string to align the generated result. The end attribute is set to /n by default.
```

Output:

```
1*1= 1
2*1= 2 2*2= 4
3*1= 3 3*2= 6 3*3= 9
4*1= 4 4*2= 8 4*3=12 4*4=16
5*1= 5 5*2=10 5*3=15 5*4=20 5*5=25
6*1= 6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36
7*1= 7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49
8*1= 8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64
9*1= 9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

Step 2 while loop

When the condition is met, the statement block is executed cyclically. To end the loop, use break or continue.

```
# while loop
# Create the variable i.

# Set a condition for the loop.

# The value of i increases by 1 in each loop.

# Check whether the conditions are met.



# Execute continue to exit the current loop.
```

```
# Exit the current big loop.
```

Output

```
1
2
Exit the current loop.
4
Exit the current big loop.
```

3 Function and Object-oriented Experiment

3.1 Procedure

3.1.1 Function Customization

Step 1 Define a function.

Define a function to return a sequence. Each number in the sequence is the sum of the former two numbers (Fibonacci sequence).

```
# Position parameter.

# Create a list to store the sequence value.

# Cycle num-2 times.


# Append the value to the list.


# Return the list.
```

Output:

```
[0, 1, 1, 2, 3]
```

Step 2 Generate parameters.

The function can be customized to generate parameters transferred in different ways.

```
# Default parameters.

# Format the output.

# hello，world      Default parameter.

# Greetings，world        Position parameter.

# Greetings, universe          Position parameter.

# hello, Gumby        Keyword parameter.
```

Output:

```
hello, world!
Greetings, world!
Greetings, universe!
hello, Gumby!
```

3.1.2 Object Orientation

Step 1 Create and use classes.

Create the Dog class.

Each instance created based on the Dog class stores the name and age. We will give each dog the **sit ()** and **roll_over ()** abilities.

```
class Dog():
"""A simple attempt to simulate a dog"""

    """Initialize the name and age attributes."""



    """Simulate a dog sitting when ordered."""


    """Simulate a dog rolling over when ordered."""
```

Output:

```
Husky is now sitting
Husky rolled over!
```

Step 2 Access attributes.

```
class Employee:



# Create the first object of the Employee class.

# Create the second object of the Employee class.
```

Output:

```
Name :   Zara ,Salary:   2000
Name :   Manni ,Salary:   5000
Total Employee 2
```

Step 3 Inherit a class.

One of the main benefits of object-oriented programming is code reuse, which is achieved through inheritance. Inheritance can be understood as the type and subtype relationships between classes.

```
class Parent:          # Define the parent class.



# Instantiate a child class.

# Invoke the method of a child class.

# Invoke the method of a parent class.

# Invoke the method of the parent class again to set the attribute value.

# Invoke the method of the parent class again to obtain the attribute value.
```

Output:

```
Invoke a child class to construct a method.
Invoke the method of a child class.
Invoke the method of a parent class.
Parent attribute: 200
```

Step 4 Class attributes and methods.

```
class JustCounter:



# An error is reported, indicating that the instance cannot access private variables.
```

Output:

```
1
2
2
```

4 Standard Library Usage

4.1 Procedure

4.1.1 Standard Library Usage

Step 1 sys

sys.exit([n]): This method can be used to exit the current program. If the value of **n** is **0**, the program exits normally; if the value of **n** is not **0**, the program exits abnormally.

```
import sys
```

Output:

```
0
1
2
3
4
5
An exception has occurred, use %tb to see the full traceback.
```

sys.path: returns the search paths of Python modules.

```

```

Output:

```
['D:\\python3.6\\python36.zip',
 'D:\\python3.6\\DLLs',
 'D:\\python3.6\\lib',
 'D:\\python3.6',
 '',
 'D:\\python3.6\\lib\\site-packages',
 'D:\\python3.6\\lib\\site-packages\\IPython\\extensions',
 'C:\\Users\\xxx\\.ipython']
```

sys.platform: returns the system running platform.

```

```

Output:

```
'win32'
```

sys.argv: transfers parameters from outside of the program to the program. The parameters are transferred in list format. The first parameter is the current file name.

Create the .py file **test.py** (in the current folder or on the desktop) and write the following code:

```

```

Switch to the file path in the command line and run the program.

```

```

Output:

```
C:\Users\[redacted]\Desktop>python test.py hello
```

Figure 4-1 Receiving parameters from outside

Step 2 os

```
import os
# os.getpid() Obtain the current process ID.

# os.getppid(): Obtain the ID of the current parent process.

# os.getcwd(): Obtain the current path.



# os.chdir(path): Change the current working directory.



# os.listdir(): Return all files in the directory.

# os.walk(): Export all files in the current path.
```

Output:

```
ID of the current process: 8792
 ID of the current parent process: 3280
 The current path is: C:\Temp\python
The modified path is: C:\
Files in the current directory: ['.ipynb_checkpoints', 'Untitled.ipynb']
C:\Temp\python\.ipynb_checkpoints\Untitled-checkpoint.ipynb
C:\Temp\python\Untitled.ipynb
C:\Temp\python\.ipynb_checkpoints
```

Figure 4-2 Execution result of the OS module

os.path module: obtains file attributes.

```
import os
# os.path.abspath(path): Return the absolute path.



# os.path.exists(path): If the file exists, True is returned; if the file does not exist, False is returned.


# os.path.getsize(path): Return the file size. If the file does not exist, an error is returned.


# os.path.isfile(path): Check whether the path is a file.


# os.path.isdir(path): Check whether the path is a folder.
```

Output:

```
The absolute path of text.txt is D:\python project\text.txt
Whether text.txt exists: True
Size of the text.txt file: 0
Whether text.txt is a file: True
Whether text.txt is a folder: False
```

Step 3 time

```
import time
# time.time(): Obtain the current timestamp.



# time.localtime(): Obtain the time tuple.



# time.asctime(): Obtain formatted time.



#time.strftime(format[, t]): Receive the time tuple and return the local time expressed in a readable
string, in the format specified by the format parameter.
```

Output:

```
Timestamp: 1555950340.4777014
Local time: time.struct_time(tm_year=2019, tm_mon=4, tm_mday=23, tm_hour=0, tm_min=25,
tm_sec=40, tm_wday=1, tm_yday=113, tm_isdst=0)
Local time: Tue Apr 23 00:25:40 2019
2019-04-23 00:25:40
```